

DPP- 01

CS & IT

Algorithm

Heap Algorithms

Q1 Which of the following is valid min heap?

- (A) 34, 38, 35, 39, 40, 36, 50
 (B) 25, 14, 12, 13, 10, 8, 16
 (C) Both (a) and (b)
 (D) Neither (a) nor (b)

Q2 The minimum number of interchanges needed to convert the array into a max-heap is

- 89, 19, 40, 17, 12, 10, 2, 5, 7, 11, 6, 9, 70
 (A) 0 (B) 1
 (C) 2 (D) 3

Q3 Consider a complete binary tree where the left and right subtrees of the root are max -heaps. The lower bounds for the number of operations to convert the tree to a heap is

- (A) $\Omega(\log n)$ (B) $\Omega(1)$
 (C) $\Omega(n \log n)$ (D) $\Omega(n^2)$

Q4 Consider a max heap represented by the array :

40, 30, 20, 10, 15, 16, 17, 8, 4

Array 1 2 3 4 5 6 7 8 9

Value 40 30 20 10 15 16 17 8 4

Now consider that a value 35 is inserted into the heap. After insertion new heap is : -

- (A) 40, 30, 20, 10, 15, 16, 17, 8, 4, 35
 (B) 40, 35, 20, 10, 30, 16, 17, 8, 4, 15
 (C) 40, 30, 20, 10, 35, 16, 17, 8, 4, 15
 (D) 40, 35, 20, 10, 15, 16, 17, 8, 4, 30

Q5 The number of possible min -heap containing each value from {1, 2, 3, 4, 5, 6, 7} exactly once.

Q6 Consider the following statements : -

I. The smallest element in a max heap is always at a leaf node.

II. The second largest element in a max heap is always a child of the root node when Max heap tree contain unique elements.

III. A max-heap can be constructed from a binary search tree in $O(n)$ time.

IV. A binary search tree can be constructed from a max-heap in $O(n)$ time.

Which of the above statements are TRUE?

- (A) I, III and IV (B) II, III and IV
 (C) I, II and III (D) I, II and IV

Q7 Let H be a binary min-heap consisting of n elements implemented as an array. What is the worst case time complexity of an optimal algorithm to find the maximum element in H?

- (A) $\theta(\log n)$
 (B) $\theta(n \log n)$
 (C) $\theta(n)$
 (D) $\theta(1)$

Q8 Which of the following is/are correct?

- (A) Bubble sort is stable but not inplace sorting technique
 (B) Insertion sort is stable sorting technique.
 (C) Selection sort is a stable sorting technique.
 (D) Bubble sort is a inplace sorting technique

Q9 Consider the following array

70	60	20	50	40	5	19	21
----	----	----	----	----	---	----	----

To sort this array using selection sort what is the result after 3rd pass?

- (A) 70, 60, 50, 40, 20, 5, 19, 21
 (B) 5, 19, 20, 21, 40, 70, 60, 50



(C) 5, 20, 19, 50, 40, 70, 60, 21

(D) 5, 19, 20, 50, 40, 70, 60, 21

Q10 Consider the following array:

50	70	13	9	85	21	16
----	----	----	---	----	----	----

How many inversions are present for above array ?

Q11 Consider the following array:

50	70	13	9	85	21	16
----	----	----	---	----	----	----

How many comparisons are needed to sort the above array using insertion sort?



Answer Key

Q1 A
Q2 C
Q3 B
Q4 B
Q5 80
Q6 C

Q7 C
Q8 B, D
Q9 D
Q10 12
Q11 16



[Android App](#)



[iOS App](#)



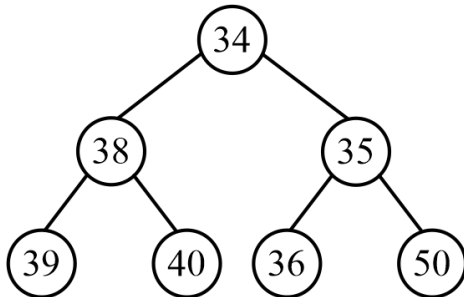
[PW Website](#)

Hints & Solutions

Note: scan the QR code to watch video solution

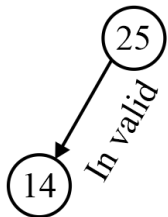
Q1 Text Solution:

(a)

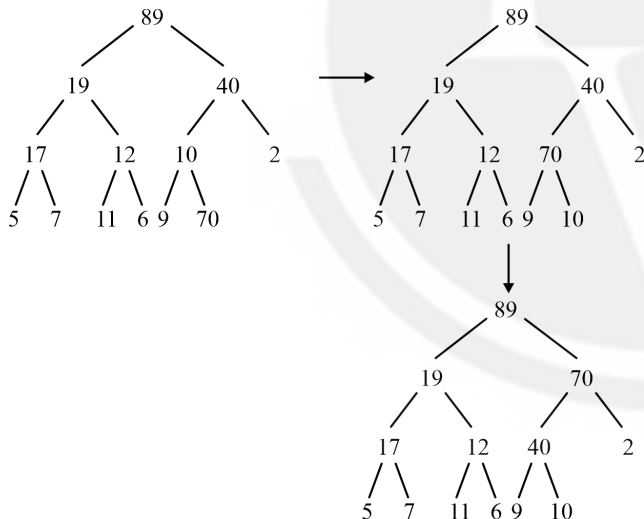


So option a) is valid.

(b)



Q2 Text Solution:



Therefore 2 swaps are need to convert into max heap.

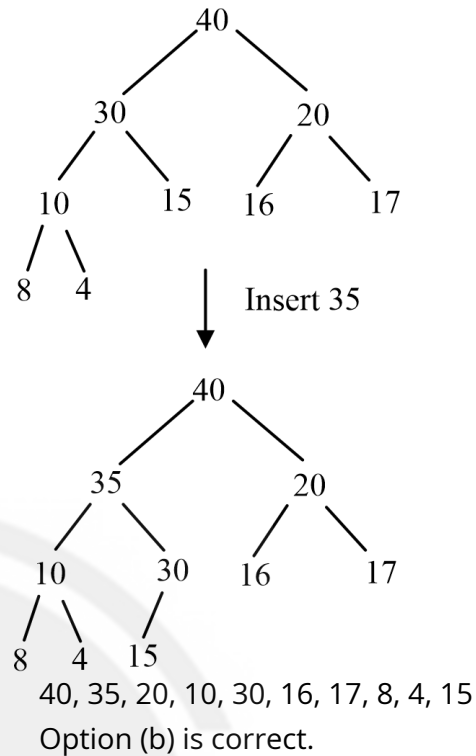
Option (c) is correct.

Q3 Text Solution:

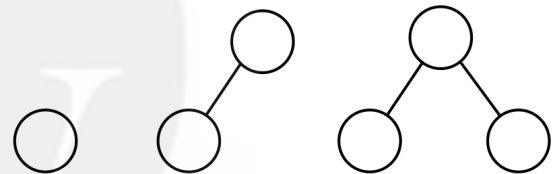
For adjusting the root node in best case we just have to compare with one node thus it will take constant time.

Lower bond for operations is $\Omega(1)$.

Q4 Text Solution:



Q5 Text Solution:



$$T(1) = 1$$

$$T(2) = 1$$

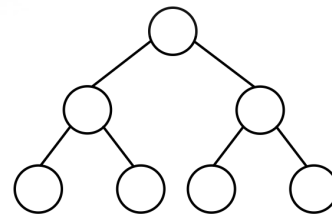
$$T(3) = 2$$

For number of min heaps :-

$$T(n) = \sum_{k=1}^{n-1} C_k \cdot T(k) \cdot T(n-k-1)$$

Where, k = number of elements in left subtree.

For '7' elements



3-elements in left and 3 elements in right.

$$k = 3$$

$$T(7) = {}^6C_3 T(3) \cdot T(7 - 3 - 1)$$

$$= {}^6C_3 T(3) \cdot T(3)$$

$$= 20 \cdot 4$$

$$= 80$$



Android App

iOS App

PW Website

Q6 Text Solution:

Binary search tree can not be constructed from max heap in $O(n)$. This statement is only false because it takes $O(n \log n)$ time.

Option (c) is correct.

Q7 Text Solution:

Let 'H' be a binary min heap.

To find maximum element we have to look at leaf nodes.

Almost there are $(n/2)$ leaf nodes.

Time complexity will be $O(n)$.

Alternative Approach: -

Convert min heap to max heap root element will be maximum element. This conversion is done with heapify method and it takes $O(n)$ time.

option (c) is correct.

Q8 Text Solution:

• Bubble sort is stable but not inplace sorting technique **False**

• Insertion sort is stable sorting technique.

True

• Selection sort is a stable sorting technique.

False

• Bubble sort is a inplace sorting technique

True

Q9 Text Solution:

Take first minimum and swap it to the first place.

Result after 3rd pass = 5, 19, 20, 50, 40, 70, 60, 21

Q10 Text Solution:

if $i < j$ && $a[i] > a[j]$

$50 = 13, 9, 21, 16, = 4$

$70 = 13, 9, 21, 16, = 4$

$13 = 9 = 1$

$9 = = 0$

$85 = 21, 16 = 2$

$21 = 16 = 1$

$16 = = 0$

Number of inversion = 12

Q11 Text Solution:

Input 50 70 13 9 85 21 16

Pass-1: 50 | 70 13 9 85 21 16 → 1 No. of comparisons

Pass-2: 50 70 | 13 9 85 21 16 → 2

Pass-3: 13 50 70 | 9 85 21 16 → 3

Pass-4: 9 13 50 70 | 85 21 16 → 1

Pass-5: 9 13 50 70 85 | 21 16 → 4

Pass-6: 9 13 21 50 70 85 | 16 → 5

9 13 16 21 50 70 85

Final sorted output

Total number of comparisons

$= 1 + 2 + 3 + 1 + 4 + 5$

$= 6 + 10 = 16$



[Android App](#)

| [iOS App](#)

| [PW Website](#)